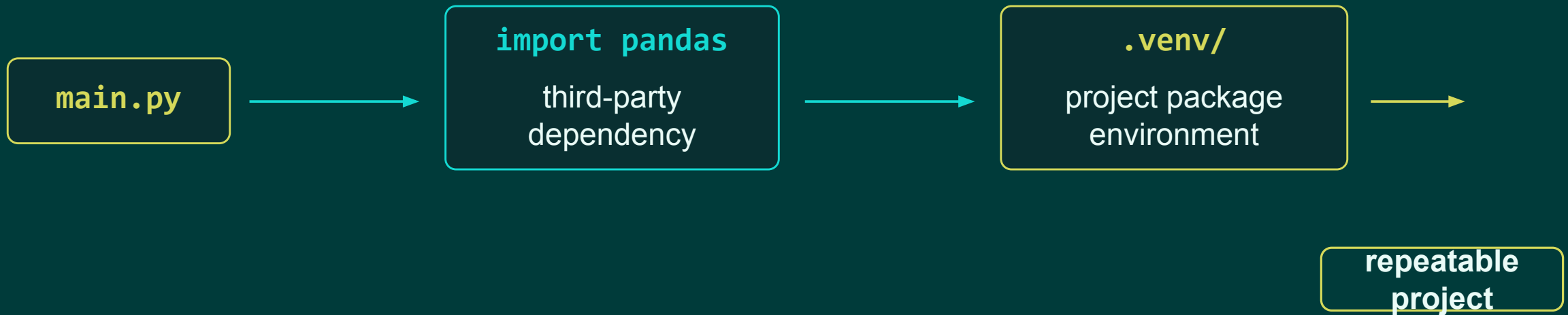


Packages, Dependencies, and Virtual Environments

Day 5 • Block 39 • 30 minutes teaching + 30 minutes exercises



Key shift

A Python project is not only your `.py` files. It also depends on the environment that runs them.

What Students Should Be Able to Do

This block explains what must be installed, documented, and isolated.

1 Distinguish standard-library modules from third-party packages

2 Explain what a project dependency is

3 Use pip conceptually to install packages

4 Explain why versions can change behavior

5 Describe what a virtual environment isolates

6 Create and read a simple requirements.txt file

Session shape

30 minutes: concepts and examples. 30 minutes: inspect imports and document dependencies.

Why This Matters

A project that runs on your laptop may fail on someone else's laptop.

```
# main.py

import csv
import json
import pandas

print("Processing records...")
```

Computer A pandas installed

Computer B ModuleNotFoundError

Mental model

The code is only one part. Installed packages are part of the project's runtime environment.

Standard Library Versus Third-Party Packages

Some imports come with Python. Others must be installed separately.

Standard library

Usually included with Python

```
import csv
import json
from pathlib import Path
from datetime import date
```

Third-party packages

Usually installed into an environment

```
import pytest
import pandas
import requests
```

Do not put standard-library modules in requirements.txt.

What Is a Dependency?

If your program needs something outside itself to run, the project depends on it.



Definition

A dependency is a package your project needs in order to run, test, or build correctly.

Importing a Package Does Not Install It

import only asks Python to load code that is already available.

```
# This line does not download anything  
import pandas
```



**If pandas is not
installed:**

ModuleNotFound
Error

```
# Installing is a separate step  
python -m pip install pandas
```

Then import can succeed because the package exists in the environment.

pip Installs Python Packages

Use `python -m pip` so the pip command belongs to the Python interpreter you are using.

```
python -m pip install package_name
```



Python environment

Installed packages become available for imports inside that environment.

Examples

```
python -m pip install pytest  
python -m pip install pandas
```

Today's goal

Understand what this command means.
Memorizing package-management details is not the point.

Which Imports Become Dependencies?

A requirements file documents third-party packages, not normal standard-library imports.

```
import csv
import json
from pathlib import Path
import pytest
import pandas
```



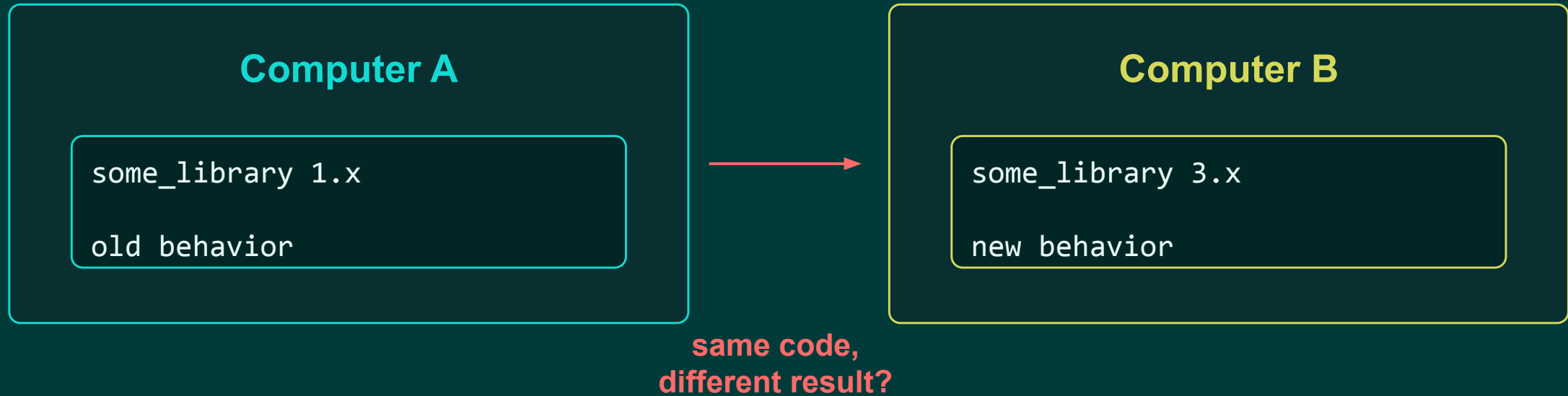
requirements.txt

```
pytest
pandas
```

csv, json, and pathlib already come with Python. pytest and pandas do not.

Versions Matter

Two computers may have the same package name but different behavior.



Professional habit

Document expected package versions so another person can recreate the same environment.

“Works on My Machine” Is an Environment Problem

The code may be correct, but the surrounding environment may be different.

Python version

3.11 vs 3.13

Installed packages

package missing

Package versions

old vs new
behavior

Operating system

path or shell
differences

```
# The project needs more than code
source files
+ package list
+ setup instructions
+ data/config
```

Reproducibility means another machine can rebuild the same working setup.

A Virtual Environment Is Project-Level Isolation

Each project can have its own installed packages.

Without isolation

Computer-wide package collection

Project A and Project B share the same packages.

With virtual environments

Project A → Environment A

Project B → Environment B

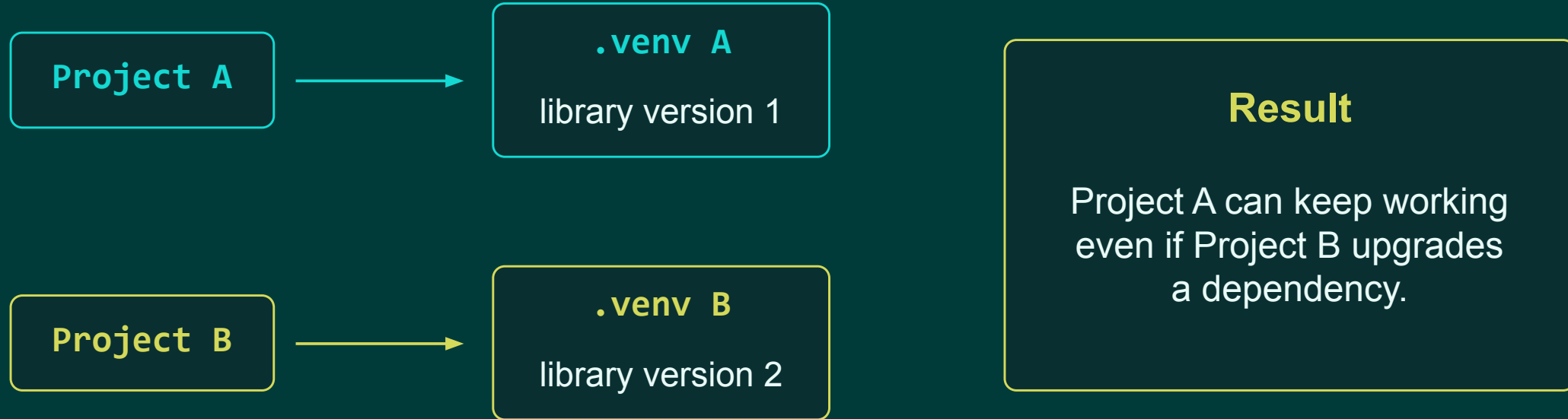
Each project can manage its own package versions.

Mental model

A venv is a separate toolbox for one project.

Virtual Environments Prevent Package Collisions

Different projects may need incompatible package versions.



Creating a Virtual Environment

The common command creates a .venv folder for the project.

```
python -m venv .venv
```



```
project/  
  .venv/  
  main.py  
  requirements.txt
```

.venv is a folder

It contains the project's environment data and installed packages.

You usually do not edit files inside it by hand.

Activation Means “Use This Environment”

Activation changes which Python and packages your terminal uses.

Before activation

Terminal uses the default Python environment.



After activation

Terminal uses this project's .venv.

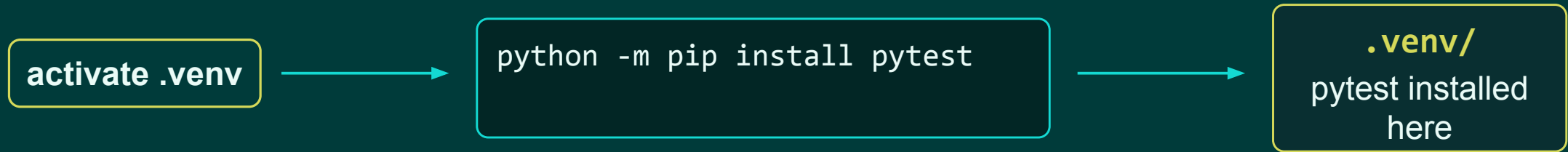
```
# Activation command depends on  
platform/shell  
# Instructor demonstrates the classroom  
setup.
```

Concept first

The exact activation command matters less than understanding that the active environment controls imports.

Install Packages Into the Active Environment

The active environment is where pip places packages.



If the wrong environment is active, packages may be installed somewhere else.

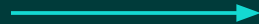
This is why `python -m pip` is preferred: it links pip to the Python interpreter being used.

requirements.txt Documents Third-Party Packages

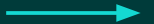
It gives another person a list of packages to install.

requirements.txt

```
pytest  
pandas  
requests
```



```
python -m pip install -r requirements.txt
```



.venv/

packages installed

Key idea

requirements.txt does not contain your code. It describes package requirements for the project.

Flexible Versus Pinned Versions

Version choices trade convenience for reproducibility.

Flexible

```
pytest  
pandas
```

Allows newer compatible versions, but another computer may install something different.

Pinned

```
pytest==9.0.0  
pandas==3.0.0
```

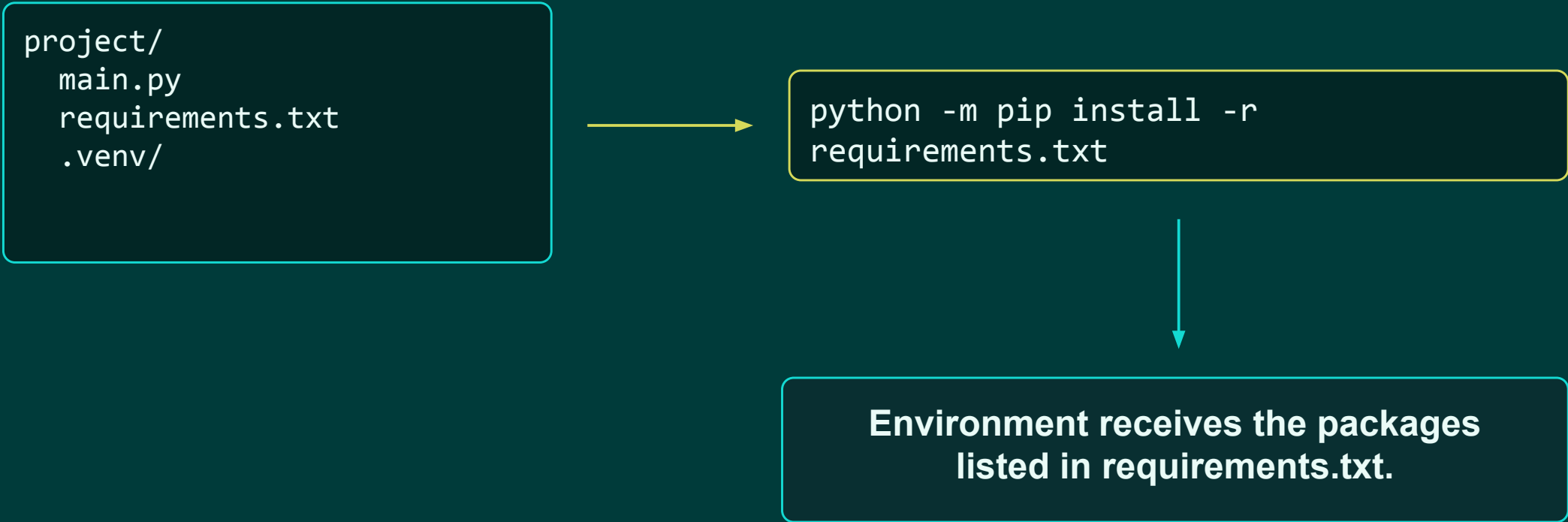
Requests exact versions, improving repeatability when versions are known.

Example version numbers are placeholders; classroom projects should use versions from the actual environment.

Installing Requirements Rebuilds the Environment

This is the “make my project environment look right” step.

```
project/  
  main.py  
  requirements.txt  
  .venv/
```



```
python -m pip install -r  
requirements.txt
```

**Environment receives the packages
listed in requirements.txt.**

A Reproducible Project Includes More Than .py Files

Another developer needs enough information to rebuild the working setup.

Source code

main.py and modules

Dependencies

requirements.txt

Configuration

config files

Data samples

raw/test data

Instructions

how to run

Tests

proof of behavior

Capstone connection

The final project should be understandable, runnable, and testable by someone else.

Inspect Imports Before Writing requirements.txt

Classify each import before adding anything to dependency documentation.

```
from pathlib import Path
import csv
import json
import pytest
import pandas as pd
from datetime import date
```



Ask for each import

Did this come with Python?
Or did we install it?

```
# requirements.txt
pytest
pandas
```

Common Environment Mistakes

These are normal beginner problems and are usually fixable.

Installing globally

Package works today but may affect other projects.

Missing requirements

Coworker cannot recreate the setup.

Ignoring versions

Different machines install different behavior.

Wrong environment active

pip installs somewhere unexpected.

Listing stdlib modules

requirements.txt says csv or json.

Committing .venv

Project folder becomes huge and machine-specific.

Rule

Document third-party dependencies. Keep project environments isolated. Do not hand-edit .venv internals.

Guided Exercise: Identify and Document Dependencies

30 minutes • inspect a project and build a requirements file.

```
data_processor/  
  main.py  
  tests/  
    test_processing.py  
  requirements.txt
```

```
# main.py  
import csv  
import json  
import pandas as pd  
  
# test_processing.py  
import pytest
```

Exercise goal

Separate standard-library imports from third-party packages and document only what must be installed.

Exercise 1: Classify Imports

Timing: 7 minutes

7 minutes

```
import csv
import json
from pathlib import Path
import pytest
import pandas
from datetime import date
```

Task

Create two lists:

1. Standard library imports
2. Third-party imports

Do not use requirements.txt yet. First decide what belongs there.

Exercise 2: Create requirements.txt

Timing: 6 minutes

6 minutes

requirements.txt

pytest
pandas

Task

Add only third-party packages. Leave csv, json, pathlib, and datetime out.

Checkpoint

Explain to a partner why each line belongs in the file.

Exercise 3: Environment Reasoning Scenarios

Timing: 9 minutes

9 minutes

Scenario A

Project A upgrades a package. Project B suddenly fails.

Scenario B

A coworker clones the project and sees `ModuleNotFoundError`.

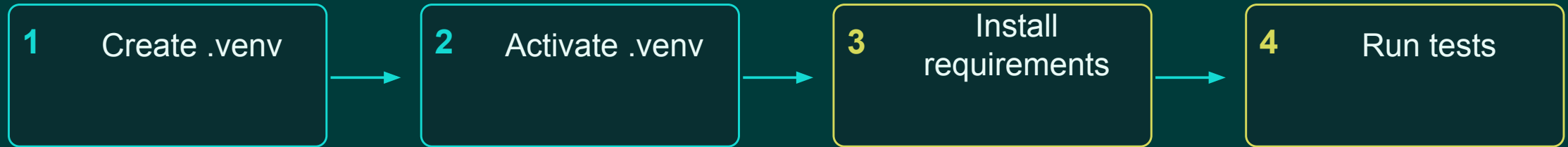
Questions:

1. What practice reduces this risk?
2. What file tells them what to install?

Exercise 4: Explain the Setup Flow

Timing: 8 minutes

8 minutes



```
python -m venv .venv
# activate .venv
python -m pip install -r requirements.txt
pytest
```

Your exact activation command may differ by operating system and shell.

Applied Challenge: Document the Day 5 Project

Add dependency documentation to the file-based processing project.

Required work

Inspect imports in every .py file.

Classify standard-library versus third-party imports.

Create requirements.txt.

Explain how another student would recreate the environment.

```
day5_project/  
  main.py  
  requirements.txt  
  src/  
  tests/  
  data/  
  config/
```

Success check

requirements.txt includes only packages that must be installed separately.

Knowledge Check

Students should be able to answer these before moving on.

What is the difference between standard library and third-party?

What is a dependency?

Why can package versions matter?

What does a virtual environment isolate?

What belongs in requirements.txt?

Why might ModuleNotFoundError happen?

Transition

Next, students will organize source code, data, configuration, tests, and dependencies into a complete project layout.